Delphi-lite: which movies will a user rate on their next active day?

Ragnar Björn and Ólafur Sær

Introduction to Deep Learning, Fall 2026 · Final project: Delphi-lite, learning from movie histories

Question. We apply Delphi, a transformer for event histories with continuous time, to MovieLens rating histories and ask: *given everything a user has rated up to the end of an active day, which movies will they rate on their next active day, and does their own history help beyond knowing what is popular at that time?*

1 Data, splits and pitfalls

MovieLens-25M [1] contains 25,000,095 ratings by 162,541 users of 59,047 movies, entered between 1995 and 2019. Every rating is one event, whatever its star value. The GroupLens licence allows research use with citation and forbids redistribution, so the hand-in contains download code and no data. We split *users* first, 80/10/10 (130,033/16,254/16,254, seed 42). The vocabulary is the 10,000 movies rated by the most training users (each by ≥88), covering 97.6% of training ratings; validation and test reuse it unchanged. Our time unit is the UTC calendar day. Pitfalls we handle: (i) timestamps are dates on which ratings were *entered*, not watched: 57% of ratings are entered on a user's first day (the sign-up batch); (ii) ratings on the same day have no meaningful order, so they are ties, stored in random order and reshuffled during training; (iii) 56% of users rate everything on one day and leave no earlier day to predict from, so a history is *eligible* if it has ≥2 active days (56,650 training, 7,067 validation, 7,129 test users); (iv) a UTC day can cut an evening in two: 28% of test targets fall one day after the last activity. Metadata: 412 titles have no year, 5,062 movies no genre, 98 titles are duplicated (we show their movieId), and 902 ratings predate the release year in the title.

2 Task and how we measure progress

For every eligible validation and test user we draw one *target day* T uniformly from their active days after the first. The *prediction time* is the end of the last active day before T; the *input* is the user's ratings on earlier days only (the latest 256); the *target* is the set of movies rated on T (median 7); the *candidates* are all 10,000 movies except those the user rated before T (MovieLens users never re-rate a movie). The metric is recall@20 = hits in the top 20 divided by $\min(20, |\text{target}|)$ [4], averaged over users, with 95% intervals; differences between methods are paired over the same users. All decisions use validation users; the test users were scored once, at the end. Baselines, fitted on training users only: POP (all-time rating counts); RECENT-POP (counts in the W days up to the prediction day; W and the counting rule were chosen on validation among 12 variants, giving $W = 30$); and ITEM-KNN (summed cosine co-occurrence with the visible history [5]), a personalised reference that ignores time.

3 Model: Delphi for movie days

Delphi [2] is a GPT-2 model (built on nanoGPT [3]) whose position embedding is replaced by a sinusoidal encoding of event time ("age"). Its loss is next-event cross-entropy plus the exponential likelihood of the waiting time, whose rate is $\sum_k e^{\text{logit}_k}$. Movies are tokens 2–10001 with their day index as age; token 0 (padding) and token 1 (Delphi's "no event") are reserved and never predicted. Ties needed care: Delphi's `mask_ties` hides same-day events from attention, but the current token still reaches its own prediction through the residual connection, which a unit test confirms. We instead read every prediction for day d from the last position on an *earlier* day, so training and evaluation both use strictly earlier days. Delphi's generation defaults (termination token 1269 "death", an 85-year horizon) are health-specific; we do not generate. Training uses random 257-event windows of eligible training users (AdamW, learning rate 10^{-3} with cosine decay, batch 64, bf16); we keep the checkpoint with the best validation recall@20.

Table 1: Validation path: recall@20 (R@20) on 7,067 users and the paired difference to RECENT-POP (± 95% interval). Grey: not kept.

Step	Change (one at a time)	R@20	Δ vs RECENT-POP
POP	all-time popularity	0.117	−0.026 ± 0.004
RECENT-POP	popularity in the last 30 days	0.143	–
ITEM-KNN	co-occurrence with the history, no time	0.133	−0.010 ± 0.005
M1 Delphi	4 layers × 128, age in years, 10k it.	0.145	+0.002 ± 0.004
M2	M1, age encoding in days	0.142	−0.001 ± 0.005
M3	M1, every position predicts its next day	0.145	+0.002 ± 0.006
M4	M1, day transitions weighted equally	0.145	+0.001 ± 0.006
M5 final	M1, trained for 30k iterations	0.155	+0.012 ± 0.005
M6	M1, 6 layers × 256 (3.5× parameters)	0.143	+0.000 ± 0.004
M7	M1 + popularity offset $\beta \log(1 + c_{30})$	0.147	+0.003 ± 0.004
M8	M5 + popularity offset	0.153	+0.010 ± 0.004

4 From the baseline to the final model (validation)

Each run in Table 1 changes one thing and is judged on validation users. Calendar time matters: RECENT-POP beats POP by 0.026, and the time-blind ITEM-KNN lies in between. A small Delphi (M1: 4 layers, 128 dimensions, 2.1M parameters) only matches RECENT-POP. The age encoding's time unit, an objective that trains every position, equal weights per day transition and a 3.5× larger model changed nothing (M2–M4, M6). Training three times longer did (M5). We also tried a *popularity offset*: adding $\beta \log(1 + c_{30})$, the movie's recent training-user count, to every log-rate makes it a baseline hazard, so the transformer only models the user's deviation from current popularity. It speeds up early training but brings no gain at convergence: the learned β fell from 0.49 (M7, 10k iterations) to 0.31 (M8, 30k), and M8 stayed below M5. The final model is therefore M5, retrained with two more seeds.

5 Results on the held-out test users

Delphi reaches recall@20 = 0.158 ± 0.005 against 0.149 for RECENT-POP, 0.136 for ITEM-KNN and 0.117 for POP (Figure 1a). The paired difference to RECENT-POP is +0.009 ± 0.005 (6% relative), and recall@10 (0.137 vs 0.130), recall@50 (0.223 vs 0.211) and NDCG@20 (0.132 vs 0.124) agree. The three training seeds give 0.157 ± 0.001 (sd), and each seed's paired interval against RECENT-POP lies above zero (lowest bound +0.002). *Where does the gain come from?* With only its most recent rating visible, Delphi already scores 0.152, level with RECENT-POP: from the date and one movie it has learned what is popular at that time. More of the user's own history adds the rest: 0.157 with 8 ratings and 0.159 with 32, flat beyond (Figure 1b). ITEM-KNN depends on history far more (0.090 with one rating). Two controls check that the gain is personal. Shuffling the order of the user's own ratings costs only −0.003 ± 0.002, so *which* movies matter more than their order. Giving the model another test user's history (the one with the closest prediction day not after the case's own) lowers recall@20 to 0.141 (−0.017 ± 0.004), below RECENT-POP: the model personalises to the history it is given. A third check concerns the user split:

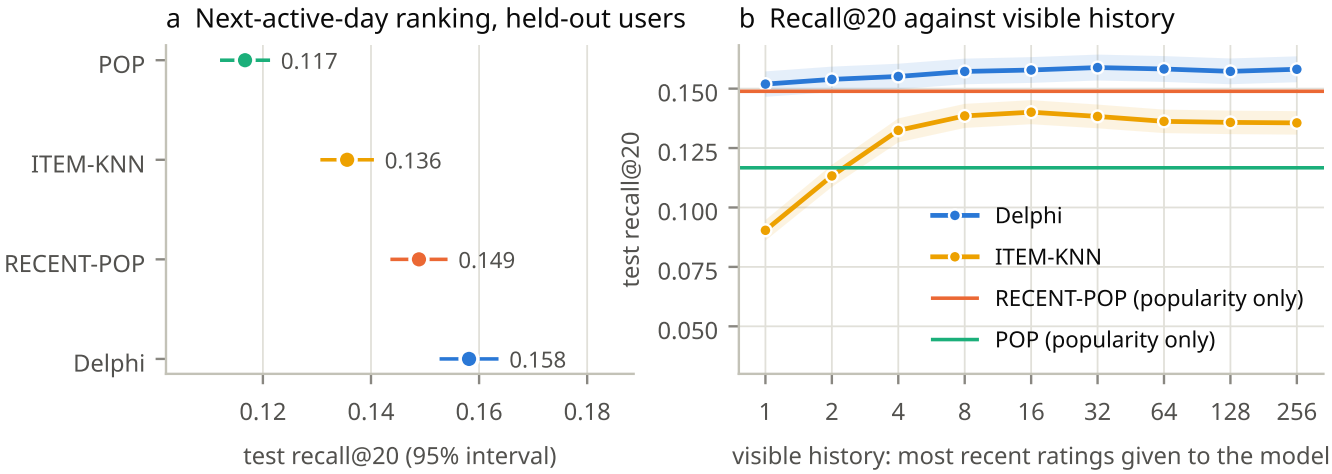


Figure 1: Test results on 7,129 held-out users. (a) Recall@20 with 95% intervals; Delphi is the final model (seed 1; seeds 1–3 span 0.156–0.158). (b) Recall@20 when Delphi or ITEM-KNN sees only the k most recent ratings before the target day; the popularity baselines use the history only to exclude rated movies.

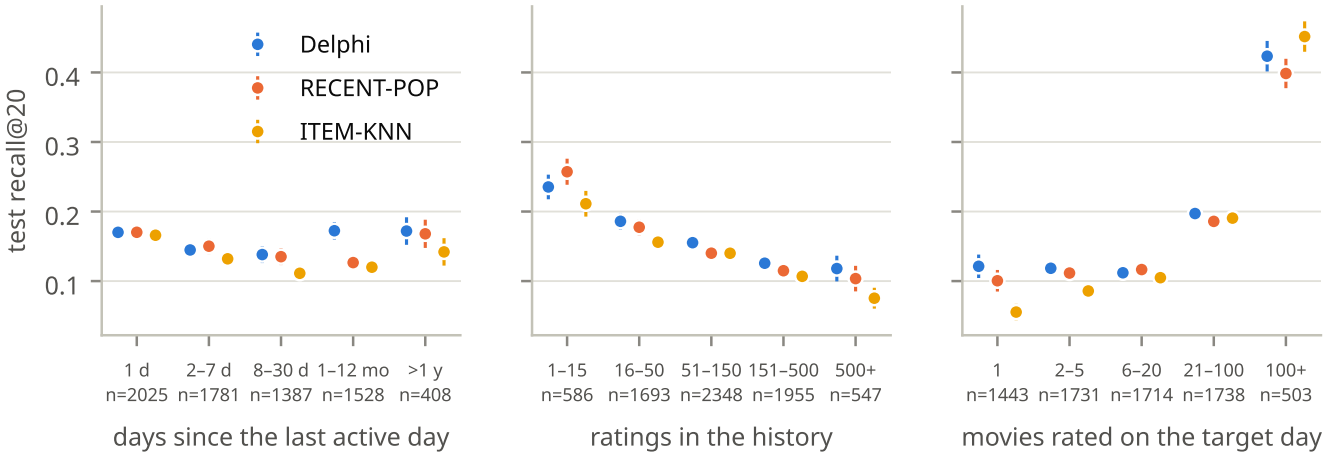


Figure 2: Where it fails: test recall@20 (95% intervals) by days since the last active day, number of ratings in the history, and number of movies rated on the target day; n = test cases per bucket.

Delphi learned from training users' later ratings, so it can rank films that appeared after a held-out user's prediction day (5% of its top 20). Restricting every method to movies some training user had rated by the prediction day leaves Delphi at 0.158 (+0.009 ± 0.005 over RECENT-POP, every seed above zero); only 2% of its hits were such films.

6 Error analysis: where and why it fails

Short histories. Recall falls with history length for every method, probably because heavy users have already rated the popular movies. Delphi's edge over RECENT-POP grows with history: with 1–15 ratings RECENT-POP is better (0.257 vs 0.235), with more than 500 Delphi leads 0.118 to 0.104 (Figure 2). **Gaps.** Next-day returns are a tie (0.170 each); the largest gain comes when a user returns after one to twelve months (0.172 vs 0.127; 0.160 vs 0.127 without later films): after such a gap the user's own taste predicts better than the popularity at their last visit. **Calendar.** Delphi wins clearly in 1996–99 (0.204 vs 0.161) but is slightly behind in 2015–19 (0.123 vs 0.129), which holds 31% of test cases. In that period users mostly rate older films (median 11.5 years after release, against 6.0 before 2015); why the model loses there is unclear. **Long tail.** Only 16% of the target movies are among the 100 most-rated. Delphi finds 32% of those, but 4% of the 46% ranked 101–1000 and almost none beyond, just like the baselines. In 42% of test cases neither Delphi nor RECENT-POP finds a single target movie. These are mostly small days (75% have at most five movies) of less popular films (55% of their targets rank outside the top 1,000, against 29% elsewhere).

7 Extras, limitations and next steps

Extras. Three seeds (above). *Data size*: with M1's budget (10k iterations), 10%, 30% and 100% of training users give 0.144, 0.147 and 0.147 on test: at that budget more users barely help; longer training does (M5). *Cost*: 2.1M parameters, 29 minutes of training on a laptop RTX 5080 (shared with another run), 3 ms per user on a CPU and 7,230 users/s on the GPU. 16 unit tests cover leakage, ties, batching, metrics and baselines; one reproduces the `mask_ties` leak. **Limitations.** Ratings are not viewings, so a sensible suggestion can count as a miss. A UTC day can split an evening session, making some next-day targets easy. The split is by user, so the model and the baselines learned from training users' ratings made after a test user's prediction day; the check in Section 5 suggests this matters little here, but a split by time as well would be stricter. Each user contributes one target day, rare and very recent movies are outside the vocabulary, and we evaluated which movies, not when. **Next steps:** a split by time, the waiting-time head, metadata inputs, synthetic histories from training users.

References

[1] F. M. Harper, J. A. Konstan. The MovieLens datasets: history and context. *ACM TiiS* 5(4):19, 2015. https://doi.org/10.1145/2827872

[2] A. Shmatko, A. W. Jung, K. Gaurav et al. Learning the natural history of human disease with generative transformers. *Nature*, 2025. https://doi.org/10.1038/s41586-025-09529-3; code: https://github.com/gerstung-lab/Delphi (MIT licence, commit fb72166).

[3] A. Karpathy. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

[4] D. Liang, R. G. Krishnan, M. D. Hoffman, T. Jebara. Variational autoencoders for collaborative filtering. *WWW*, 2018.

[5] B. Sarwar, G. Karypis, J. Konstan, J. Riedl. Item-based collaborative filtering recommendation algorithms. *WWW*, 2001.